Information Retrieval 4

Traditional Retrieval Models

Makoto P. Kato, National Institute of Informatics / University of Tsukuba

Based on: Jimmy Lin, Rodrigo Nogueira, Andrew Yates. *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies, Springer, 2021.

Review of the Previous Lecture: Relevance and Search Evaluation

Relevance = the relationship between a document and an information need. Inherently subjective, but relative comparisons between systems are stable

Ranking metrics = AP for binary relevance, nDCG for graded relevance. The choice of metric depends on the task

Test collections = three components: corpus + topics + qrels. In recent years, the setup of training on MS MARCO and evaluating on TREC DL is widely used

This lecture covers the mathematical frameworks for "how to estimate relevance" -- **retrieval models** -- covering four approaches. We trace the history of IR: Boolean → Vector Space → Probabilistic → Language Model

How do we estimate the relevance between a query and a document? We study four different approaches to this question. Each model has fundamentally different views on "what relevance is"

First Half: Boolean Model & Vector Space Model

- What is a retrieval model?
- Boolean model: exact matching based on set theory
- Vector Space Model: similarity between vectors
- Principles of TF-IDF weighting

Second Half: Probabilistic Model & Language Model

- Probability Ranking Principle (PRP)
- Binary Independence Model and its evolution to BM25
- Language model: the probability that a document "generates" a query
- The relationship between smoothing and IDF

What Is a Retrieval Model?

Retrieval Model: Given a query q and a document d, a **mathematical framework** for how to estimate the relevance between them

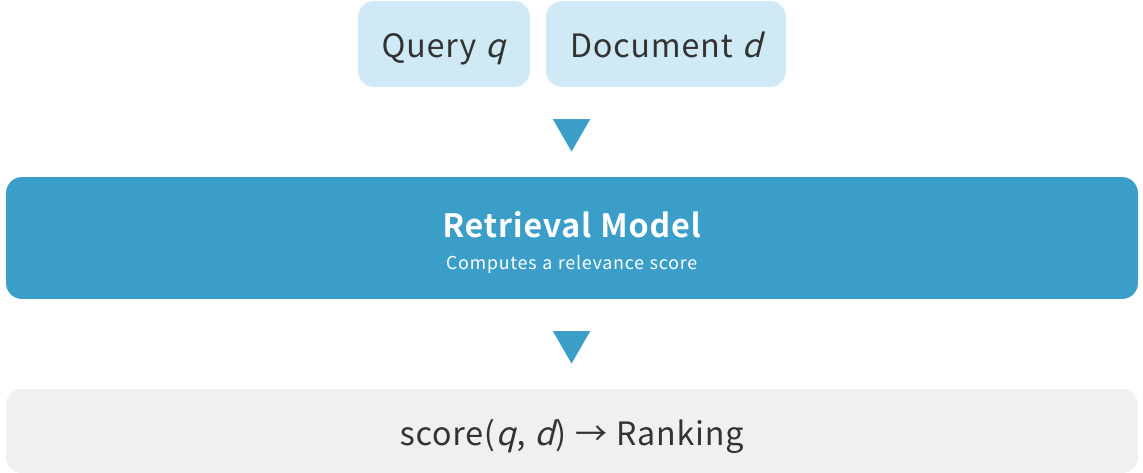
Different retrieval models have fundamentally different views on "what relevance is" and "how to compute it"

The evaluation metrics learned previously (Precision, AP, nDCG, etc.) are **applicable to any retrieval model**

Each model has a different mathematical foundation:

- Set theory, linear algebra, probability theory, statistical language models

Common Structure of Retrieval Models



Historical Genealogy of the Four Models

1950s-

Boolean

Exact matching based on set theory. No ranking

→

1960s-

Vector Space

Similarity between vectors. Partial matching and ranking possible

→

1970s-

Probabilistic

Probabilistic estimation of relevance. Evolved into BM25

→

1990s-

Language Model

Probability that a document "generates" a query

Each model evolved to **overcome the limitations** of the previous model

Boolean → "Cannot rank" → Vector Space Model

Vector Space → "Weak theoretical foundation" → Probabilistic Model

Probabilistic → "An alternative with a different modeling direction" → Language Model

Current neural IR is also an extension of these traditional models

Boolean Model

The most classical and simplest retrieval model

Queries are expressed as **Boolean logic expressions** (AND, OR, NOT)

Binary judgment of whether a document satisfies the logical expression (relevant or not relevant)

No ranking of relevance (all returned documents are "equally relevant")

Formulation of the Boolean Model

Document: a set of index terms $d = \{t_1, t_2, \ldots\}$

Query: a Boolean logic expression (e.g., $t_1 \wedge (t_2 \vee \neg t_3)$)

Results: the set of documents satisfying the logical expression

Boolean Operators

Operator	Meaning	Example
AND	Contains both	A AND B
OR	At least one	A OR B
NOT	Does not contain	A NOT B

A powerful tool for experts (librarians, patent searchers, etc.). Search results are logically explainable

Manning et al., "Introduction to Information Retrieval", Cambridge University Press, 2008, Ch.1

Corpus: 3 documents

d_1
"information retrieval system"
{information, retrieval, system}

d_2
"information filtering method"
{information, filtering, method}

d_3
"retrieval and ranking"
{retrieval, ranking}

Query	Results	Reason
`information AND retrieval`	$\{d_1\}$	Only d_1 contains both
`information OR retrieval`	$\{d_1, d_2, d_3\}$	Contains at least one
`information AND NOT filtering`	$\{d_1\}$	Contains information but not filtering

Problem: `information AND retrieval AND ranking` → zero results. No document contains all three terms. Yet d_1 and d_3 are partially related -- the Boolean model cannot express this "partial matching"

Set-Theoretic Interpretation -- Venn Diagram

Each index term t defines a set D_t of documents in the corpus containing that term

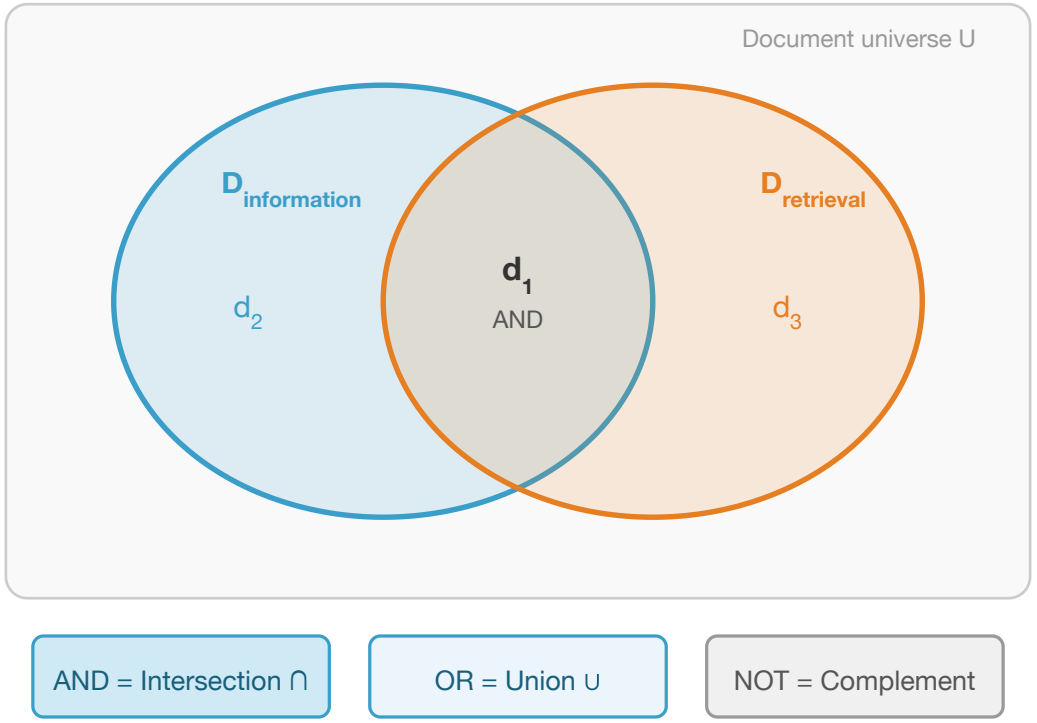
Boolean operations = execution of **set operations**

- AND: $D_{t_1} \cap D_{t_2}$ (intersection)
- OR: $D_{t_1} \cup D_{t_2}$ (union)
- NOT: $\overline{D_t}$ (complement)

Efficiently implementable using an inverted index

- Each term's posting list = set D_t
- AND = enumeration of common elements in the lists

Set-Theoretic Interpretation of Boolean Operations



Advantages and Limitations of the Boolean Model

Advantages

- Intuitive and easy to understand
- Search results are logically explainable
- Efficiently implementable with inverted index
- Powerful for experts (patent and legal search)

Limitations

- **No ranking**: results are either too many or too few
- **No partial matching**: missing one AND term means not relevant
- **Difficult query construction**: Boolean logic is hard for general users
- **Ignores term importance**: all terms are treated equally

The Vector Space Model was proposed to overcome the Boolean model's limitations of "no ranking" and "no partial matching"

Vector Space Model

Vector Space Model (VSM)

Proposed by **Salton, Wong, and Yang [1975]**

Both documents and queries are represented as high-dimensional vectors

Relevance is estimated by the **cosine similarity** between vectors

Unlike the Boolean model, **partial matching and ranking** are possible

Formulation of the Vector Space Model

Vocabulary $V = \{t_1, t_2, \ldots, t_{|V|}\}$

Document vector: $\vec{d} = (w_{t_1,d}, \ldots, w_{t_{|V|},d})$

Query vector: $\vec{q} = (w_{t_1,q}, \ldots, w_{t_{|V|},q})$

Relevance = similarity between vectors

Evolution from Boolean to VSM

Boolean Model

Term presence or absence (0 or 1)
→ Binary: relevant or not relevant

▼

Vector Space Model

Considers term **weights** (0.0 to ∞)
→ Continuous relevance scores

Salton, Wong, and Yang, "A Vector Space Model for Automatic Indexing", CACM, 1975

Cosine Similarity

$$\mathrm{sim}(\vec{q}, \vec{d}) = \cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| \cdot |\vec{d}|} = \frac{\sum_{t \in V} w_{t,q} \cdot w_{t,d}}{\sqrt{\sum_{t \in V} w_{t,q}^2} \cdot \sqrt{\sum_{t \in V} w_{t,d}^2}}$$

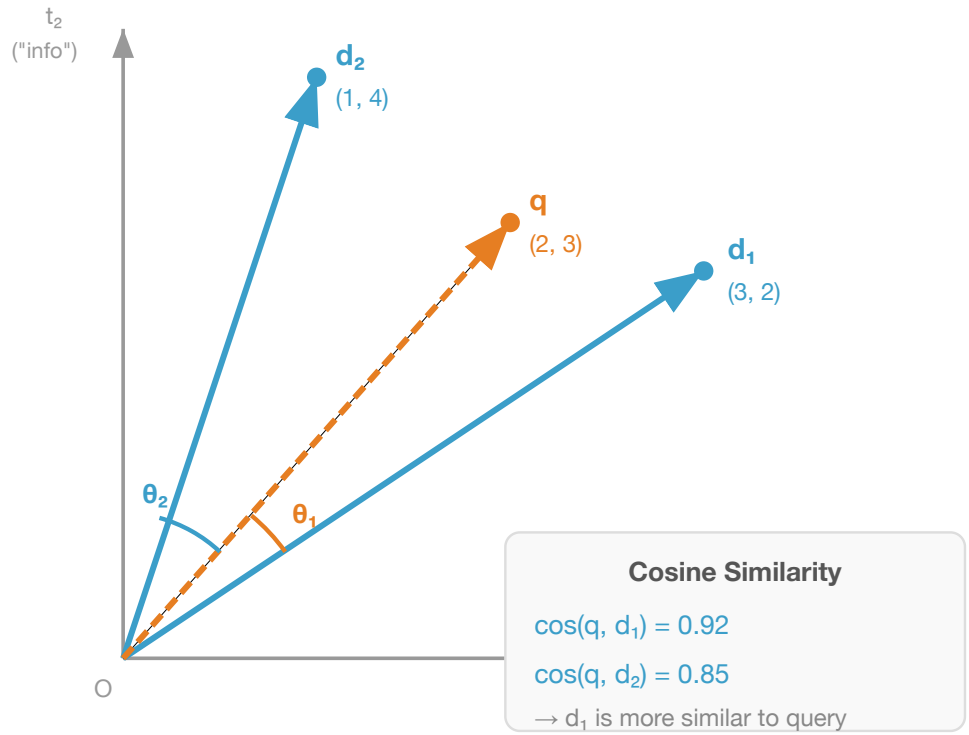
Range: $[0, 1]$ (when weights are non-negative)

Closer to 1 means more similar → higher relevance

Built-in normalization that is **independent of vector length (document length)**

In practice, only the overlap between query terms and document terms needs to be computed (efficient due to sparse vectors)

Geometric Interpretation of the Vector Space Model



Smaller θ (closer vector directions) → higher similarity

How should the **weight** $w_{t,d}$ of each dimension in the vector be set?

TF (Term Frequency):

A term appearing frequently in a document → the document is likely related to that term

However, raw TF is problematic: a term appearing 10 times is not 10 times more important

→ Use **sublinear TF scaling** to achieve saturation

$$\text{tf-weight}(t,d) = \begin{cases} 1 + \log(\text{tf}(t,d)) & \text{if } \text{tf}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

Raw TF vs Log TF

tf	Raw TF	Log TF
1	1	1.00
2	2	1.69
5	5	2.61
10	10	3.30
100	100	5.61

The logarithm causes the weight increase to **diminish** as the occurrence count grows. This concept of "saturation" is also carried forward to BM25

Luhn, "The Automatic Creation of Literature Abstracts", IBM J. Res. Dev., 1958

IDF (Inverse Document Frequency):

Terms appearing in many documents have **low discriminative power** (e.g., "the," "is," "a")

Terms appearing in only a few documents are **important clues**

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)}$$

N: total number of documents in the corpus

$\mathrm{df}(t)$: number of documents containing term t

TF-IDF weight:

$$w_{t,d} = \text{tf-weight}(t, d) \times \mathrm{idf}(t)$$

IDF Intuition (N = 10,000)

Term	df	IDF
"the"	9,500	0.02
"retrieval"	1,000	2.30
"BM25"	50	5.30
"LambdaMART"	5	7.60

TF-IDF intuition: Assigns high weight to terms that are frequent within the document (high TF) and rare across the entire corpus (high IDF)

Sparck Jones, "A Statistical Interpretation of Term Specificity and Its Application in Retrieval", Journal of Documentation, 1972

TF-IDF Calculation Example

Setup: $N = 10{,}000$ documents, query q = "information retrieval"

Weight Computation for Document d_1

"information": tf = 5, df = 1,000

"retrieval": tf = 3, df = 100

IDF:

idf(information) = log(10000/1000) ≈ **2.30**

idf(retrieval) = log(10000/100) ≈ **4.61**

TF-IDF weights:

w(information) = (1+log5) × 2.30 ≈ **6.00**

w(retrieval) = (1+log3) × 4.61 ≈ **9.68**

Interpretation of Weights

"retrieval" receives a **higher weight** than "information"

Reason: "retrieval" is **rarer** across the corpus (df = 100 vs 1,000) → higher IDF

This difference in weights determines ranking quality

Luhn [1958]'s term frequency and Sparck Jones [1972]'s IDF were integrated into TF-IDF

TF Weight Saturation -- Why Linear Does Not Work

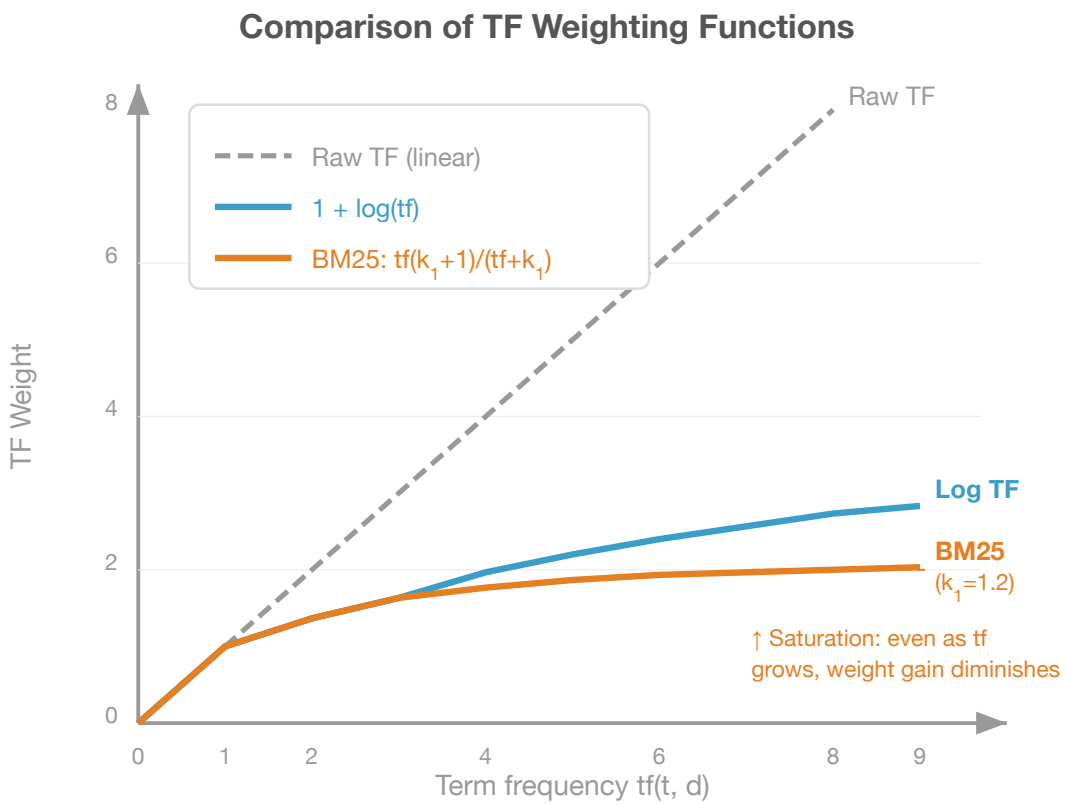
The more frequently a term appears, the more important it is, but **linear proportionality is inappropriate**

- "retrieval" appears 10 times → 10x more important than 1 time? → **NO**
- Importance diminishes (law of diminishing returns)

Three TF weighting schemes:

- **Raw TF**: tf as-is (linear) -- overly favors long documents
- **Log TF**: 1 + log(tf) -- standard in VSM
- **BM25 TF**: $tf(k_1+1)/(tf+k_1)$ -- asymptotically approaches (k_1+1)

BM25's saturation function is **theoretically derived** from the probabilistic model



BM25's TF component asymptotes to (k_1+1). 10 occurrences are not 10x as important as 1

Robertson et al., "The Probabilistic Relevance Framework: BM25 and Beyond", FnTIR, 2009

Advantages and Limitations of the Vector Space Model

Advantages

- **Partial matching**: documents containing only some query terms still get a score
- Can return **ranked results**
- Intuitive geometric interpretation
- Relatively easy to implement

Limitations

- **Term independence assumption**: cannot handle synonyms or polysemy
- **Bag-of-words**: ignores word order
- **Weak theoretical foundation**: thin justification for why cosine similarity is a good estimate of relevance
- Document length normalization may be insufficient

The probabilistic model provides the theoretical justification for "why does this similarity measure reflect relevance?"

Probabilistic Model

Probabilistic Model

Probability Ranking Principle (PRP)
[Robertson, 1977]

Returning documents in **decreasing order of the probability of relevance** maximizes expected retrieval effectiveness

$$\text{Optimal ranking} = P(R = 1 \mid q, d) \text{ in decreasing order}$$

R: a random variable representing relevance ($R = 1$: relevant, $R = 0$: not relevant)

PRP is a **foundational principle** in IR -- the theoretical pillar of probabilistic models

The challenge is how to estimate $P(R = 1 \mid q, d)$

Difference from VSM

Vector Space Model

Estimates relevance via cos(q, d)
Theoretical basis: "vectors that are close are related" (assumption)

Probabilistic Model

Estimates relevance via P(R=1|q,d)
Theoretical basis: probability theory + PRP (principled justification)

Robertson, "The Probability Ranking Principle in IR", Journal of Documentation, 1977

Two assumptions of **BIM** [Robertson and Sparck Jones, 1976]:

1. **Binary**: only the presence/absence of terms in documents (frequency is ignored)
2. **Independence**: term occurrences are mutually independent

Mathematical formulation:

Represent document d as a binary vector $\vec{x} = (x_1, \ldots, x_{|V|})$

Compute the odds ratio using Bayes' theorem:

$$O(R \mid \vec{x}, q) = \frac{P(\vec{x} \mid R=1, q)}{P(\vec{x} \mid R=0, q)} \cdot \underbrace{\frac{P(R=1 \mid q)}{P(R=0 \mid q)}}_{\text{constant}}$$

Apply the independence assumption to decompose into a product over terms

Reasoning Flow of BIM

Want to estimate relevance probability P(R=1|d,q)

▼ Bayes' theorem

Convert to likelihood ratio

P(d|R=1,q) / P(d|R=0,q)

▼ Independence assumption

Sum of per-term weights c_i → RSV

Robertson and Sparck Jones, "Relevance Weighting of Search Terms", JASIS, 1976

BIM Derivation (1): Bayes' Theorem and Odds Ratio

Goal: We want to use $P(R = 1 \mid \vec{x}, q)$ for ranking. Direct estimation is difficult → transform using the **odds ratio**

Step 1 -- Applying Bayes' theorem:

$$P(R = 1 \mid \vec{x}, q) = \frac{P(\vec{x} \mid R = 1, q) \cdot P(R = 1 \mid q)}{P(\vec{x} \mid q)}$$

Step 2 -- Converting to odds ratio ($P(\vec{x} \mid q)$ cancels):

$$O(R \mid \vec{x}, q) = \frac{P(R = 1 \mid \vec{x}, q)}{P(R = 0 \mid \vec{x}, q)} = \frac{P(\vec{x} \mid R = 1, q)}{P(\vec{x} \mid R = 0, q)} \cdot \underbrace{\frac{P(R = 1 \mid q)}{P(R = 0 \mid q)}}_{\text{prior odds (constant)}}$$

A monotonic transformation of the odds ratio does not change the ranking, so the **prior odds can be ignored** . What remains is only the **likelihood ratio** $\frac{P(\vec{x}|R=1,q)}{P(\vec{x}|R=0,q)}$

Manning et al., "Introduction to Information Retrieval", Cambridge University Press, 2008, § 11.3

Step 3 -- Naive Bayes independence assumption: assume term occurrences are mutually independent

$$\frac{P(\vec{x} \mid R=1,q)}{P(\vec{x} \mid R=0,q)} = \prod_{i=1}^{|V|} \frac{P(x_i \mid R=1,q)}{P(x_i \mid R=0,q)}$$

Step 4 -- Each term is binary ($x_i \in \{0,1\}$), so separate into present and absent terms:

$$= \prod_{x_i=1} \frac{p_i}{q_i} \cdot \prod_{x_i=0} \frac{1-p_i}{1-q_i}$$

where $p_i = P(x_i = 1 \mid R=1,q)$, $q_i = P(x_i = 1 \mid R=0,q)$

Step 5 -- Assume $p_i \approx q_i$ for non-query terms (non-query terms do not differ between relevant and non-relevant):

→ Only **terms in the query** contribute:

$$= \prod_{\substack{x_i=1 \\ t_i \in q}} \frac{p_i}{q_i} \cdot \prod_{\substack{x_i=0 \\ t_i \in q}} \frac{1-p_i}{1-q_i}$$

Manning et al., "Introduction to Information Retrieval", Cambridge University Press, 2008, § 11.3.2

Step 6 -- Take the logarithm to convert products into sums:

$$\log O(R \mid \vec{x}, q) = \sum_{\substack{x_i=1 \\ t_i \in q}} \log \frac{p_i}{q_i} + \sum_{\substack{x_i=0 \\ t_i \in q}} \log \frac{1-p_i}{1-q_i} + \text{const}$$

Step 7 -- Reorganize the "$x_i = 0$ terms": The sum over absent terms can be rewritten as a subtraction from the sum over all query terms. Collecting document-independent terms into a constant:

$$\text{RSV}(q, d) = \sum_{t_i \in q \cap d} c_i$$

where **RSJ weight** c_i:

$$c_i = \log \frac{p_i(1-q_i)}{q_i(1-p_i)} = \underbrace{\log \frac{p_i}{1-p_i}}_{\text{odds in relevant docs}} - \underbrace{\log \frac{q_i}{1-q_i}}_{\text{odds in non-relevant docs}}$$

The essence of RSJ weights: the difference between "the odds of term t_i appearing in relevant documents" and "the odds of appearing in non-relevant documents." Terms with a larger difference have higher discriminative power

Robertson and Sparck Jones, "Relevance Weighting of Search Terms", JASIS, 1976

When relevant documents are unknown (initial retrieval):

When relevance information r, R is unavailable, the following approximation is used:

Assumption: $p_i = 0.5$ (probability of a term appearing in relevant documents is unknown → fifty-fifty)

$$\log \frac{p_i}{1 - p_i} = \log \frac{0.5}{0.5} = 0$$

q_i is approximated by $\mathrm{df}(t)/N$:

$$c_i \approx \log \frac{1 - q_i}{q_i} = \log \frac{1 - n/N}{n/N} = \log \frac{N - n}{n}$$

When $n \ll N$:

$$c_i \approx \log \frac{N}{n} = \mathrm{IDF}$$

Important consequence

IDF is derived as a **special case** of the probabilistic model (BIM). That is:

The **IDF in TF-IDF** has a probabilistic justification

When relevance information is available, more precise RSJ weights can be used (relevance feedback)

Summary of derivation steps:
$P(R = 1|d)$ → Bayes → odds ratio → independence → product → log → **sum of RSJ weights** → approximation → **IDF**

Robertson and Sparck Jones, "Relevance Weighting of Search Terms", JASIS, 1976

RSJ Weight Derivation

For each term t_i:

$p_i = P(x_i = 1 \mid R = 1)$: probability that term t_i appears in relevant documents

$q_i = P(x_i = 1 \mid R = 0)$: probability that term t_i appears in non-relevant documents

RSJ (Robertson-Sparck Jones) weight:

$$c_i = \log \frac{p_i(1 - q_i)}{q_i(1 - p_i)}$$

Ranking score (sum over query terms only):

$$\mathrm{RSV}(q, d) = \sum_{t_i \in q \cap d} c_i$$

RSV = Retrieval Status Value

Terms not in the query do not contribute

Intuition Behind RSJ Weights

Term t_i has a **high** RSJ weight ⇔

It **frequently appears** in relevant documents (p_i is large)

It **rarely appears** in non-relevant documents (q_i is small)

In other words, terms with higher **discriminative power** between relevant and non-relevant are more important

This can be seen as a generalization of IDF: when relevance information is available, more accurate weights can be computed; when unavailable, it approximates to IDF (next slide)

Robertson and Sparck Jones, "Relevance Weighting of Search Terms", JASIS, 1976

For each query term t, p_i and q_i are estimated from the following **2×2 contingency table**:

	Relevant docs ($R = 1$)	Non-relevant docs ($R = 0$)	Total
t present ($x = 1$)	r	$n - r$	n
t absent ($x = 0$)	$R - r$	$(N - n) - (R - r)$	$N - n$
Total	R	$N - R$	N

N: total documents, R: relevant documents

n: documents containing term t (= df)

r: relevant documents containing term t

Estimates: $p_i = r/R$, $q_i = (n - r)/(N - R)$

Problem: During initial retrieval, relevant documents are unknown (R, r are unknown) → assume p_i to be constant 0.5 and approximate $q_i \approx n/N$:

$$c_i \approx \log \frac{N - n}{n} \approx \log \frac{N}{n}$$

This is **IDF itself**

Robertson and Sparck Jones, "Relevance Weighting of Search Terms", JASIS, 1976

From BIM to BM25 -- A Theoretical Bridge

Limitation of BIM: uses only term **presence/absence** and ignores frequency

2-Poisson Model [Harter, 1975; Robertson, 2004]:

- Term occurrence frequency is modeled as a mixture of two Poisson distributions
  - **Elite terms** (related to the document's topic): occur frequently
  - **Non-elite terms**: occur infrequently
- From this assumption, it follows that the effect of TF should be a **saturation function**

Intuition: Going from 1 to 5 occurrences adds information, but going from 50 to 55 barely changes anything

Development Path from BIM to BM25

BIM (binary)
Term presence/absence only. TF = 1 or 0

▼ 2-Poisson Model

TF Saturation
$tf(k_1+1)/(tf+k_1)$ for diminishing returns

▼ Document length effects

BM25
Add document length normalization → complete form

Robertson, "Understanding Inverse Document Frequency", Journal of Documentation, 2004; Robertson and Zaragoza, FnTIR, 2009

BM25 Formulation

Based on the 2-Poisson model, adding TF saturation and document length normalization to BIM:

$$\mathrm{BM25}(q,d) = \sum_{t \in q \cap d} \underbrace{\log \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}}_{\text{IDF term (RSJ weight approx.)}} \cdot \underbrace{\frac{\mathrm{tf}(t,d) \cdot (k_1 + 1)}{\mathrm{tf}(t,d) + k_1 \cdot (1 - b + b \cdot \frac{l_d}{L})}}_{\text{TF saturation + doc length normalization}}$$

BM25 Parameters

Parameter	Role	Typical Value
k_1	Controls TF saturation	1.2
b	Degree of doc length normalization	0.75

$k_1 = 0 \rightarrow$ degenerates to binary model (BIM)

$k_1 \rightarrow \infty \rightarrow$ approaches raw TF

$b = 0 \rightarrow$ no normalization

$b = 1 \rightarrow$ full normalization

BM25 details were covered in Lecture 1. The key point here is that the probabilistic model theory **derives IDF and TF saturation from first principles**

Robertson et al., "Okapi at TREC-3", TREC, 1994; Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", FnTIR, 2009

BM25 Scoring Structure -- Integration of Three Components

BM25 integrates three components multiplicatively:

1. **IDF**: rarer terms → higher weight
2. **TF saturation**: diminishing returns for repeated terms
3. **Length normalization**: corrects advantage of longer documents

Parameters: k_1 controls TF saturation speed (k_1=0 → BIM), b controls length normalization (b=0 → none)

BM25 Scoring Structure

$$BM25(q, d) = \Sigma_{t \in q \cap d} IDF(t) \times TF_sat(t, d)$$

IDF Component

$$\log((N - df(t) + 0.5) / (df(t) + 0.5))$$

N: Total number of documents
df(t): Number of docs containing t
→ **Rarer terms get higher weight**

×

TF Saturation

$$\frac{tf(t,d) \times (k_1+1)}{tf(t,d)+k_1(1-b+b \cdot l_d/L)}$$

k_1=1.2: TF saturation speed
b=0.75: Doc length norm.

Effect of k_1

k_1=0: binary(BIM) | $k_1 \to \infty$: raw TF

Effect of b

b=0: no normalization | b=1: full normalization

BM25 = Practical realization of the probabilistic model (BIM) theory
IDF = "discriminative power", TF saturation = "diminishing returns", b = "length fairness"

Robertson and Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", FnTIR, 2009

Advantages and Limitations of the Probabilistic Model

Advantages

- A theoretically justified framework **grounded in probability theory**
- Natural integration with relevance feedback
- BM25 is highly effective in practice → still widely used as a baseline
- Efficiently implementable with inverted index

Limitations

- **Term independence assumption** (in reality, terms have strong dependencies)
- BIM's binary assumption (partially resolved by BM25)
- Parameter estimation can be unstable (especially when relevant documents are scarce)

The language model introduced next takes a different probabilistic approach to modeling relevance

Language Model

Language Model for IR

Basic Idea of the Language Model

Introduced to IR by **Ponte and Croft [1998]**

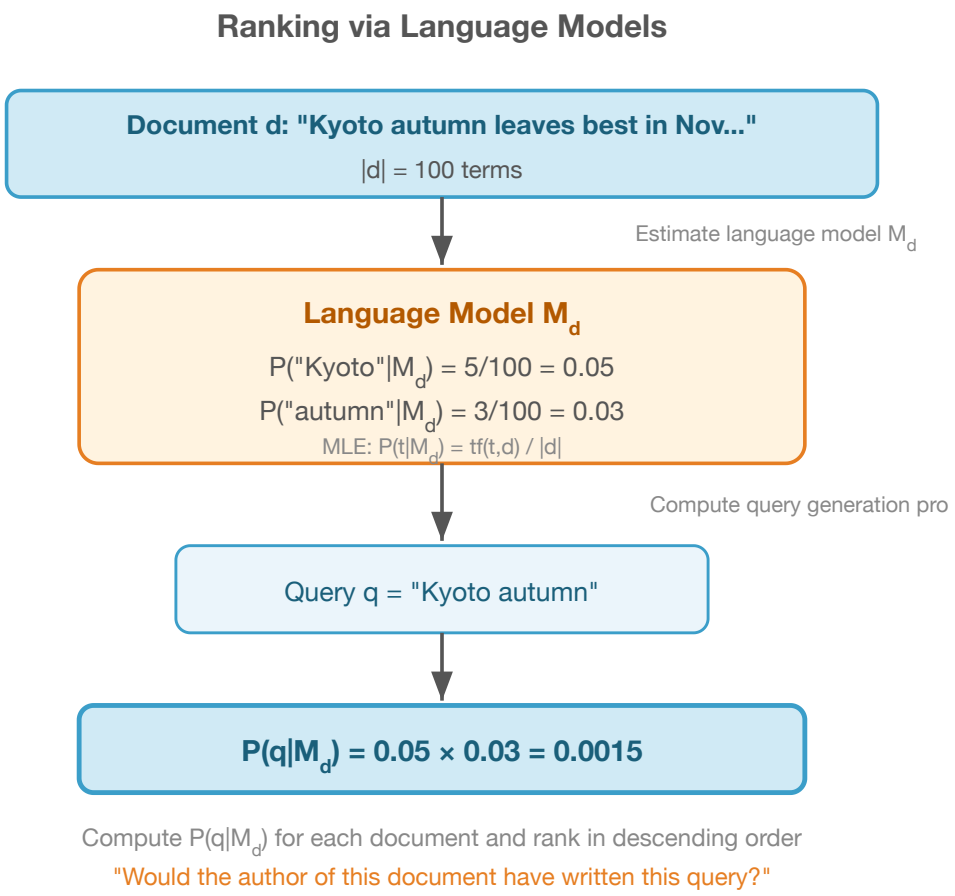
A different approach from the probabilistic model (BIM/BM25)

Language Model Approach: Assume each document d has a language model M_d, and rank by the **probability that the document's language model generates the query**

$$P(q \mid M_d) \text{ in decreasing order for ranking}$$

Metaphor: whether the author of the document would be likely to use the words in the query

A different **modeling direction** from the probabilistic model's $P(R = 1 \mid q, d)$



Ponte and Croft, "A Language Modeling Approach to Information Retrieval", SIGIR, 1998

Most basic form -- unigram language model:

View the query $q = (q_1, q_2, \ldots, q_m)$ as a sequence of terms and assume term independence:

$$P(q \mid M_d) = \prod_{i=1}^{m} P(q_i \mid M_d)$$

Take the logarithm to use as a score:

$$\log P(q \mid M_d) = \sum_{i=1}^{m} \log P(q_i \mid M_d)$$

Maximum Likelihood Estimation (MLE):

$$P_{\text{MLE}}(t \mid M_d) = \frac{\text{tf}(t, d)}{|d|}$$

Calculation Example

Document *d*: 100 words, "Kyoto" appears 5 times, "foliage" appears 3 times

$P(\text{Kyoto} \mid M_d) = 5/100 = 0.05$

$P(\text{foliage} \mid M_d) = 3/100 = 0.03$

$P(q \mid M_d) = 0.05 \times 0.03 = 0.0015$

Zero probability problem: If a query term never appears in the document, $P(t \mid M_d) = 0 \rightarrow$ the entire product becomes 0. A single missing term makes the document non-relevant

Ponte and Croft, "A Language Modeling Approach to Information Retrieval", SIGIR, 1998

Smoothing -- Solving the Zero Probability Problem

Assign **non-zero probability** to terms not appearing in the document → "mix in" probability from the collection-wide language model M_C

Jelinek-Mercer (Linear Interpolation)

$$P_{\text{JM}}(t \mid M_d) = (1-\lambda) \cdot P_{\text{MLE}}(t \mid M_d) + \lambda \cdot P_{\text{MLE}}(t \mid M_C)$$

$\lambda \in [0, 1]$: smoothing parameter

$P_{\text{MLE}}(t \mid M_C) = \text{cf}(t)/|C|$

Larger λ → stronger influence of the collection model

Typical value: $\lambda = 0.1 \sim 0.7$

Dirichlet Smoothing

$$P_{\text{Dir}}(t \mid M_d) = \frac{\text{tf}(t, d) + \mu \cdot P_{\text{MLE}}(t \mid M_C)}{|d| + \mu}$$

$\mu > 0$: pseudo-count (parameter of the prior distribution)

The **shorter** the document, the greater the effect of smoothing → **natural document length normalization**

Typical value: $\mu = 1000 \sim 2500$

Dirichlet smoothing can be interpreted as Bayesian estimation (MAP estimation with a Dirichlet prior). Shorter documents are more heavily influenced by the prior

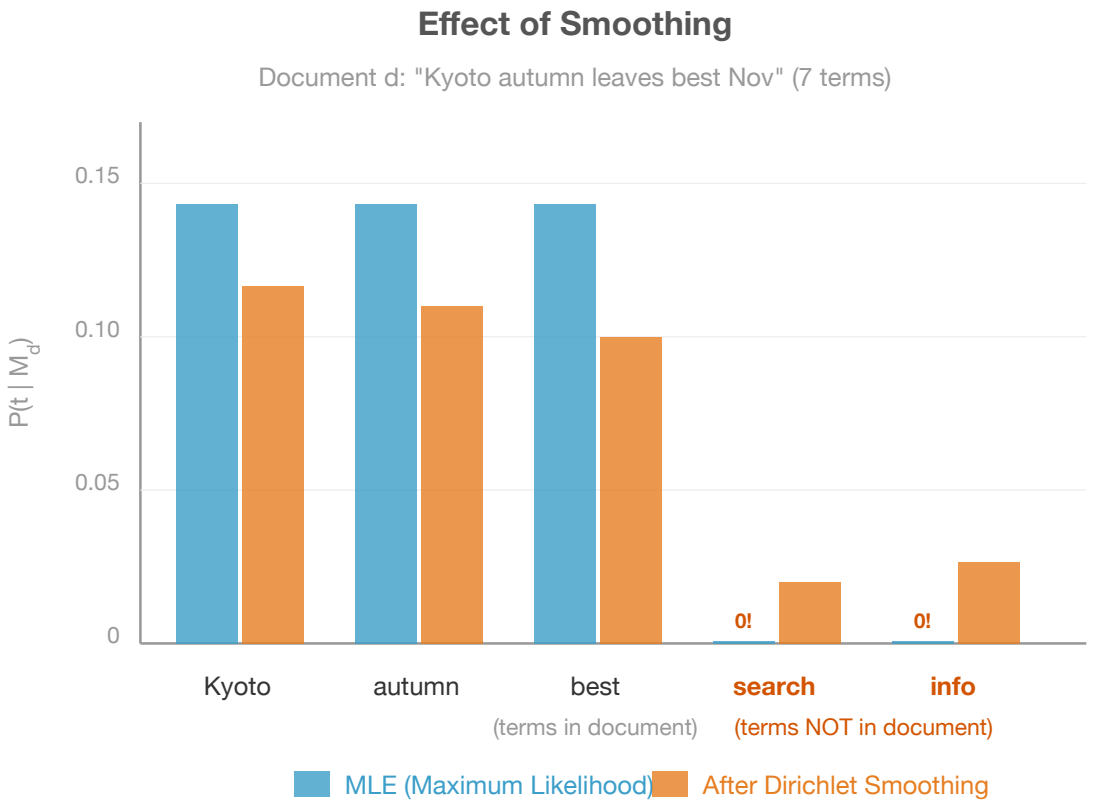
Zhai and Lafferty, "A Study of Smoothing Methods for Language Models Applied to Information Retrieval", TOIS, 2004

Effect of Smoothing -- Visualization

Without smoothing (MLE): Only in-document terms have P > 0. Missing query terms → P = 0 → **entire document non-relevant**

After Dirichlet smoothing:

- Out-of-document terms assigned probability from collection frequency
- In-document term probabilities decrease slightly (mass redistribution)
- Shorter documents affected more → **natural length normalization**



Smoothing assigns nonzero probability to unseen terms → solves the zero probability problem

Zhai and Lafferty, "A Study of Smoothing Methods for Language Models Applied to Information Retrieval", TOIS, 2004

Mathematical Role of Smoothing

Zhai and Lafferty [2004]'s analysis: smoothing serves two roles

1. **Improved estimation**: corrects inaccurate term probability estimates in short documents
2. **IDF-like effect**: terms rare across the entire collection receive higher weights

Expanding JM smoothing:

$$\log P(q \mid M_d) = \sum_{t \in q \cap d} \log \frac{P_{\text{JM}}(t \mid M_d)}{\lambda \cdot P(t \mid M_C)} + \text{constant}$$

$P(t \mid M_C)$ in the denominator → terms **rarer** in the collection have a smaller denominator → **higher weight**

This is the same effect as IDF

Explicit vs Implicit IDF

TF-IDF / BM25

Explicitly multiplies IDF
log(N / df(t))

Language Model

IDF arises **implicitly** through smoothing
1 / P(t | M_C)

From different theoretical starting points, the same "emphasize rare terms" effect is derived

Zhai and Lafferty, "A Study of Smoothing Methods for Language Models Applied to Information Retrieval", TOIS, 2004

Comparison with BM25

Aspect	Probabilistic Model (BM25)	Language Model (QL)
Modeling direction	$P(R = 1 \mid q, d)$	$P(q \mid M_d)$
Theoretical foundation	Probability Ranking Principle (PRP)	Statistical language modeling
TF handling	Saturation function (bounded)	Natural saturation via logarithm
Doc length normalization	Explicit via parameter b	Implicit via smoothing
IDF handling	Explicit IDF term	Implicit via smoothing
Parameters	k_1, b	λ or μ
Relevance feedback	Update RSJ weights	KL divergence model

In practice, performance is **comparable** : which is better depends on the task and collection. BM25 is widely adopted for its implementation simplicity and robustness, while the language model is valued for its theoretical consistency and extensibility (e.g., RM3)

Ponte and Croft, 1998; Robertson and Zaragoza, 2009

Advantages and Limitations of the Language Model

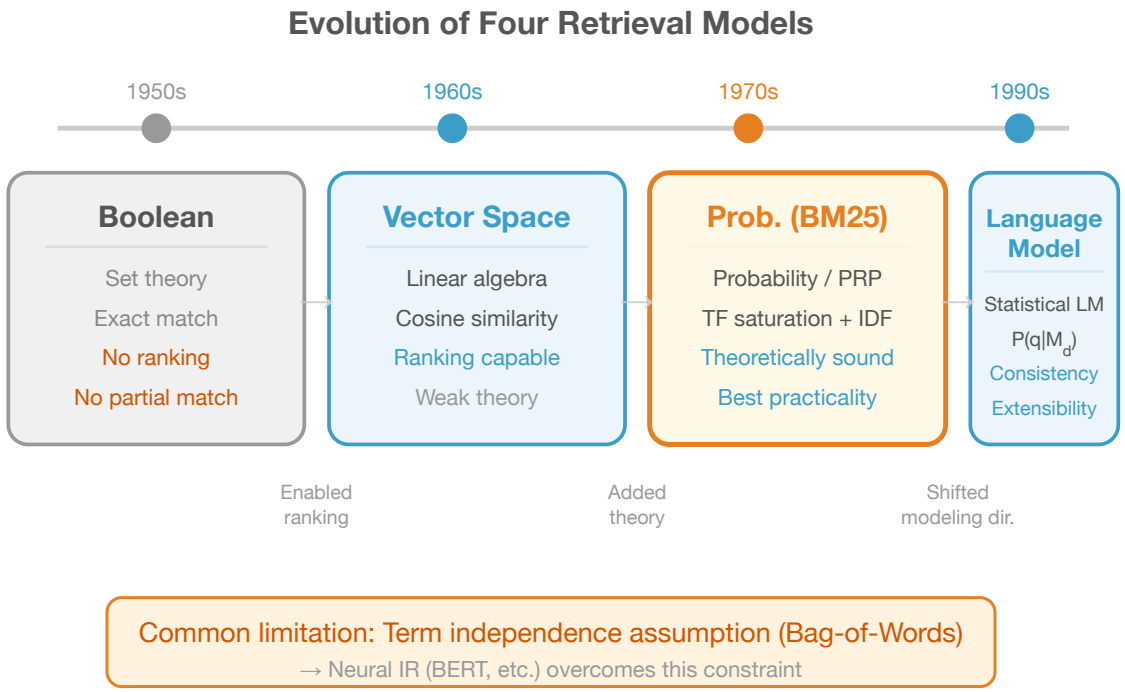
Advantages

- A theoretically consistent framework
- Smoothing **unifies TF-IDF-like weighting and document length normalization**
- Natural integration with query expansion and relevance feedback (via KL divergence model)
- Evolution to Relevance Model (RM3)

Limitations

- Term independence assumption (unigram): ignores word order and phrases
- Choice of smoothing parameter significantly affects results
- The "document generates query" direction can be counter-intuitive

Evolution of the Four Retrieval Models -- Bird's-Eye View



Each model evolved to **overcome the limitations** of the previous one

Boolean → "Cannot rank" → Vector Space

Vector Space → "Weak theoretical foundation" → Probabilistic Model

Probabilistic → "An alternative with a different modeling direction" → Language Model

Common to all models: term independence assumption (Bag-of-Words) → overcome by neural IR

Comprehensive Comparison of the Four Models

Aspect	Boolean	Vector Space	Probabilistic (BM25)	Language Model
Ranking	No	Yes	Yes	Yes
Partial matching	No	Yes	Yes	Yes
Theoretical foundation	Set theory	Geometry (weak)	Probability theory	Statistical language model
TF consideration	None	Yes	Yes (saturated)	Yes (logarithmic)
IDF consideration	None	Explicit	Explicit	Implicit
Doc length normalization	None	Cosine normalization	Parameter b	Smoothing
Independence assumption	—	Yes	Yes	Yes
Current importance	Special purposes	Educational	Very high	High

All four models share the **term independence (bag-of-words) assumption** . What goes beyond this assumption is neural IR and BERT, which we studied in Lecture 2 -- capturing semantic relationships between terms through contextual representation learning

1. **Boolean model**: exact matching retrieval based on set theory. Cannot rank or do partial matching, but logically clear
2. **Vector Space Model**: represents documents and queries as vectors, estimating relevance via cosine similarity. TF-IDF weighting is the foundation
3. **Probabilistic model**: probabilistic estimation of relevance based on the Probability Ranking Principle (PRP). The evolution from BIM's RSJ weights to BM25 is the greatest achievement
4. **Language model**: ranks by the probability that a document generates the query. Smoothing achieves TF-IDF-like weighting and document length normalization in a unified way

Looking Back at All Four Lectures

Lecture 1: Fundamentals of document ranking and keyword search with BM25

Lecture 2: Machine learning (LtR) and neural IR (BERT)

Lecture 3: The concept of relevance and search evaluation (metrics and test collections)

Lecture 4: Mathematical frameworks of four traditional retrieval models

Common limitations of traditional models:

- Term independence assumption (Bag-of-Words)
- Cannot capture semantic similarity between terms
- "retrieval" and "search" are treated as completely different terms

Topics for the next lecture:

- Fundamentals of neural networks
- **Transformer**: capturing word context with Self-Attention
- **BERT**: general-purpose language representations via pre-training
- Applying fine-tuning to ranking tasks

From the Bag-of-Words world to a world that understands context . Transformers and BERT dramatically improved search accuracy

Keywords: Self-Attention, Multi-Head Attention, Positional Encoding, BERT, MLM, Fine-tuning